

Serialization for the Parking System Application: Client/Server

ICT 4315: Object Oriented Methods and Programming II

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

May 24, 2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, M.S.

Dean: Michael J. McGuire, MLS

Design

The goal of this assignment was to replace the original line-by-line client-server protocol with a modern serialization approach. The original protocol required the client to send a command name on the first line, then one parameter=value pair per subsequent line, followed by the word "end" to signal completion. The server responded symmetrically, sending its output line-by-line and terminating with "end." While functional, this protocol is brittle, of sorts. It cannot be consumed by standard web tools, it has no structure that generic parsers can understand, and adding new fields requires careful positional parsing.

The replacement protocol uses JSON. Two new classes serve as the contract between client and server: `ParkingRequest` and `ParkingResponse`. A `ParkingRequest` holds a command name (a string such as "CUSTOMER" or "CAR") and a `Properties` object containing the command's parameters. A `ParkingResponse` holds an integer status code (200 for success, 400 for a client error, 500 for a server error) and a string message. Both classes provide a `toJson()` method that serializes the object into a single-line JSON string, and a static `fromJson()` method that deserializes it back. The client sends exactly one JSON line per request; the server replies with exactly one JSON line per response. The entire protocol now fits on a single line in each direction, which makes it curl-compatible and trivially unit-testable.

I made a deliberate design decision to avoid any external JSON library. The JSON produced and consumed by these classes is simple and structured, i.e. a top-level object with string fields and a nested flat object for parameters. It also has a hand-rolled serializer of roughly thirty lines that handles it completely. This keeps the project dependency-free beyond

JUnit, which the assignment environment already provides. The `escape` and `unescape` helpers handle the only two characters that need encoding within JSON string values: backslash and double-quote. This approach also makes the serialization logic transparent and teachable, which fits the educational context.

The `Server` class was updated to handle each client connection on a separate daemon thread. This means the server's main accept loop never blocks on I/O, and multiple clients can send commands without queuing. The `process()` method, which takes a raw JSON string and returns a `ParkingResponse`, was deliberately separated from socket I/O so it can be called directly in unit tests without a live network connection. This separation of concerns is a direct application of the Single Responsibility Principle: the `Server` class manages sockets and threads, while `process()` handles command dispatch, and `ParkingService` handles business logic.

Implementation Challenges

The first challenge I encountered was JSON parsing without a library. From my experience, a general-purpose JSON parser handles nested objects, arrays, unicode escapes, and numbers with arbitrary precision. The parking system's wire format is far simpler in the fact that there are no arrays, no nesting beyond one level, and no numeric parameter values. However, the parser still needed to handle escaped quotes inside string values, which the original `extractStringField` method in `ParkingResponse` did correctly using a character-by-character walk rather than a simple `indexOf`. Recognizing that the response's

message field frequently contains colons, pipe characters, and spaces (all of which are legal inside JSON string values) made this careful parsing necessary.

The second challenge was deciding where the customer ID flows. When a customer is registered, the server returns a message containing the UUID that was auto-generated. The client must copy that UUID and supply it when registering a car. In a GUI or web application, a session layer would handle this automatically. In a command-line tool, the user must do it manually. The updated client output explicitly labels the UUID in the message, making it easy to copy, and the usage instructions make the two-step process explicit. This also revealed a benefit of the JSON protocol: because the response is structured, a future automated client could parse the UUID from the message field programmatically rather than expecting it at a fixed position in plain text.

The third challenge for me was null and error handling. One big takeaway for me this week was that a server must never crash due to a malformed client request. The `process()` method wraps its entire body in a try-catch that returns a 500-status response for any unexpected exception, while input validation in the command classes returns 400-status errors for missing required parameters. The `ParkingRequest.fromJson()` method returns an empty request rather than throwing for null or blank input, so a dropped or truncated line results in a 400 error rather than a `NullPointerException`. This defensive design means the server stays up regardless of what the client sends.

Reflection

I feel like through this assignment, I have discovered quite a bit about serialization. For example, the JSON approach is meaningfully better than the line-by-line protocol in three ways. First, it is self-describing, meaning, any JSON parser in any language can read a `ParkingRequest` or `ParkingResponse` without knowing the custom "end" terminator convention. Second, it is testable without a running server. So, the `ServerProcessTest` class exercises the full request-to-response pipeline by calling `server.process()` directly, which was impossible with the old design. Lastly, it aligns with industry standards. A developer familiar with REST APIs will immediately recognize the structure: a request with a command and parameters, a response with a status code and message. This familiarity lowers the learning curve for any developer who works on this system in the future, in my opinion.

The main trade-off of the hand-rolled approach is fragility. The custom parser does not handle all valid JSON, instead, deeply nested objects, escaped unicode sequences, or numeric parameter values would break it. For this assignment's scope, I would think that is acceptable. In a production system, replacing the two `toJson/fromJson` methods with calls to a library such as `Gson` or `Jackson` would take under an hour and would add robustness without changing any other code, because the interface (`toJson()` returns a `String`, `fromJson()` takes a `String`) would remain identical. I tried to make the design written to make that future swap easy.

Overall, this assignment demonstrated how serialization is not just a technical detail but a fundamental part of system design to me. I think my scope on serialization was clouded before this week... it wasn't that I didn't understand it necessarily, but it wasn't something I spent a lot

of time thinking about. For example, I found that the choice of wire format determines what tools can consume an API, how errors propagate, and how easy it is to test the system in isolation. Moving from a bespoke line-by-line protocol to JSON, even a hand-rolled subset of JSON, produces a system that is more readable, more testable, and more aligned with how modern distributed systems communicate.